Google Play Billing: setup, activation, testing, pricing

The code side is finished: the official **GodotGooglePlayBilling 3.3.0** plugin (Billing Library 9.1) is in `addons/GodotGooglePlayBilling`, enabled, and its adapter (`src/autoload/store_play_billing.gd`) is wired to `Store` (BillingManager), `PurchaseProcessor` (ledger) and the shop UI. What remains is console work in Google Play.

1. Play Console: one-time

1. **Payments profile**: Play Console → Settings → Payments profile → create/link a merchant account (needs bank and tax details). Without it you cannot create paid products.
2. **Upload a build that contains the BILLING permission** (any build from this repo has it) to the **Internal testing** track. Play only lets you create products after such an upload.
3. **Create the eleven products** (step-by-step owner guide: `OWNER_SETUP_ADMOB_BILLING.md`): Monetize → Products → **One-time products** → Create product. The IDs must match exactly (they are in `data/balance.json` → `shop.products`):

Product ID	Name	Description	Type in Play	Default price
`remove_ads`	Remove Ads	Removes every ad. Video rewards are given instantly.	One-time product (the code acknowledges and never consumes it)	USD 4.99
`addon_pirate_hat`	Pirate Hat	Rocky addon (decoration, looks only)	One-time product (acknowledged, never consumed)	USD 0.99
`addon_cap`	Slugger Cap	Rocky addon (decoration, looks only)	One-time product (acknowledged, never consumed)	USD 0.99
`addon_crown`	Royal Crown	Rocky addon (decoration, looks only)	One-time product (acknowledged, never consumed)	USD 1.99
`addon_viking`	Viking Helmet	Rocky addon (decoration, looks only)	One-time product (acknowledged, never consumed)	USD 0.99
`addon_top_hat`	Top Hat	Rocky addon (decoration, looks only)	One-time product (acknowledged, never consumed)	USD 0.99
`addon_shades`	Cool Shades	Rocky addon (decoration, looks only)	One-time product (acknowledged, never consumed)	USD 0.99
`addon_specs`	Smart Specs	Rocky addon (decoration, looks only)	One-time product (acknowledged, never consumed)	USD 0.99
`coin_crate`	Coin Crate	30,000 coins	One-time product (the code consumes it, so it can be bought again)	USD 1.99

Product ID	Name	Description	Type in Play	Default price
`gem_crate`	Gem Crate	400 gems	One-time product (consumed)	USD 4.99
`hellfire_crate`	Hellfire Crate	3 Hellfire strikes: fire boulders destroy every enemy structure	One-time product (consumed)	USD 2.99

In the new Play Console each one-time product has a **purchase option** ("buy"). Leave it as the default "Buy" option, no pre-orders. Set it to **Active**. 4. Consumable vs non-consumable is decided by the code (`consume` vs `acknowledge`), not by a Play setting. Do not change the IDs or the types in `balance.json` after launch.

2. Testing purchases

1. Play Console → Settings → **License testing** → add the Gmail accounts of your testers (your own included). Licence testers are never charged.
2. Add the same accounts to the Internal testing track's tester list and open the opt-in link on the phone. Install the game **from Play** (the internal track link), not by `adb install`, at least once. Sideloaded debug builds signed with another key report "product query failed (12)", which is what we saw on the Realme. This is expected.
3. With a licence tester, the Play purchase sheet shows test cards. Run every case:

Test card	Expected in game	Covered by automated test
Test card, always approves	Reward granted once, "Purchased!" message, consumed (crates) / owned (Remove Ads)	yes
Test card, always declines	"Payment declined" message, nothing granted	yes
Slow test card, approves after a few minutes	"Payment pending…" message, nothing granted; reward arrives later (also after an app restart)	yes
Slow test card, declines after a few minutes	Pending, then nothing	yes
Back button on the sheet	"Purchase cancelled", nothing granted	yes
Kill the app right after paying	Reward granted on next launch (recovery), never twice	yes
Buy Remove Ads, uninstall, reinstall	Remove Ads restored automatically at start; "Restore Purchases" button also works	yes
Play refunds/revokes a crate	Nothing happens client side (see server section)	n/a

The automated suite (`--run-tests=billing`, 27 billing checks) runs these flows against a simulated Play backend.

3. Pricing by region

Play converts the USD price automatically, but automatic conversions produce odd prices and are too expensive for low-income markets. Set these by hand in each product → Prices → **Edit prices per country**, or create **Pricing templates** (Settings → Pricing templates) named after the four products and apply them. The numbers follow Play's own local-price suggestions rounded to "nice" local endings and adjusted for purchasing power. Check them against Play's suggestions on the day you set them (exchange rates move).

Region (currency)	Remove Ads	Coin Crate	Gem Crate	Hellfire Crate
United States (USD)	4.99	1.99	4.99	2.99
Eurozone (EUR, VAT incl.)	4.99	1.99	4.99	2.99
United Kingdom (GBP)	4.49	1.79	4.49	2.49
Canada (CAD)	6.99	2.79	6.99	3.99
Australia (AUD)	7.99	2.99	7.99	4.49
Japan (JPY)	750	300	750	450
South Korea (KRW)	6,500	2,500	6,500	3,900
India (INR)	249	99	249	149
Indonesia (IDR)	49,000	19,000	49,000	29,000
Philippines (PHP)	229	89	229	129
Vietnam (VND)	99,000	39,000	99,000	59,000
Thailand (THB)	139	55	139	79
Pakistan (PKR)	750	300	750	450
Brazil (BRL)	16.90	6.90	16.90	9.90
Mexico (MXN)	89	35	89	49
Turkey (TRY)	99.99	39.99	99.99	59.99
Egypt (EGP)	169	69	169	99
Saudi Arabia (SAR)	18.99	7.49	18.99	10.99
South Africa (ZAR)	79.99	29.99	79.99	44.99

Rules of thumb: keep the **ratio** between the four products the same everywhere (Coin Crate is the cheapest entry point, Gem Crate the most expensive), and keep Remove Ads at the Gem Crate price (it is permanent and grants every video reward instantly). Russia and Belarus currently have Play paid purchases suspended; leave them on the default price.

4. Optional server verification

The client already calls `purchase_verifier.gd` when `balance.shop.verify_url` is set: `POST {verify_url}` with JSON `{package_name, product_id, purchase_token}` and expects `{"valid": true}`. With `verify_required: true` a purchase is only granted after a valid answer.

Minimal backend (Cloud Run / Cloud Functions / any HTTPS server): 1. Google Cloud console → create a service account; Play Console → Users and permissions → invite the service-account email with "View financial data" and "Manage orders". 2. On the server call **Google Play Developer API** `purchases.products.get` (`androidpublisher/v3/applications/{package}/purchases/products/{productId}/tokens/{token}`), check `purchaseState == 0` and that the token was not seen before (store tokens in a database). 3. Return `{"valid": true}`. Keep the service-account key only on the server. Never in the game. 4. Optional: subscribe to **Real-time developer notifications** (Pub/Sub) and the **Voided Purchases API** to detect refunds of crates.

Without a server the game still works (local ledger + Play's own fraud checks); add one if fraud shows up in the order reports.

5. Google Play login

Not needed for purchases. Billing uses the Google account that is signed in to the Play Store on the phone, and Remove Ads is restored from that account on any device. **Play Games Services** (sign-in, cloud save, achievements, leaderboards) is optional. To add it later: Play Console → Play Games Services → create a game project and link it to the app, add the SHA-1 of the Play app signing key and upload key under credentials, then install the official `godot-play-game-services` plugin and wire cloud save into `Save` (the save is a single JSON file, so a snapshot upload/download is enough).

6. Go-live checklist

- [ ] Merchant account active
- [ ] Eleven products created with the exact IDs, Active, regional prices set
- [ ] Licence testers added; all test-card cases above pass on an internal-track install
- [ ] Real AdMob App ID and `ad_ids.cfg` in place (see `MONETIZATION.md`)
- [ ] Data safety form updated (`POLICY_AUDIT.md`)